

```

        // Note Dispose is not overridable. We want this
behavior for all
        // subclasses.
        public void Dispose()
        {
            Console.WriteLine("In Dispose()");
            Dispose(true);
            // We've already disposed this object; don't
call its finalizer
            GC.SuppressFinalize(this);
        }

        protected virtual void Dispose(bool disposing)
        {
            Console.WriteLine("In Dispose(bool)");
            if (disposing)
            {
                Console.WriteLine("In Dispose(bool):
disposing");

                // call dispose on managed components that
need it

                // e.g., myField.Dispose()
            }
            // now free up that socket, file, etc.
        }

        ~DisposePatternTest()
        {
            Console.WriteLine("In DisposePatternTest dtor");
            Dispose(false);
        }

        #endregion

        public static void Test()
        {
            using (DisposePatternTest t = new

```